



Week 7

✓ Thread

- Why create a thread use less resources than create a process?
- Categories of OS (Based on Thread)
- Thread States
- Thread Control Block
- Types of Threads
- Multithreading Model
- Thread Library
- Java Thread
- Joining Thread
- Java Thread States
- Thread Cancellation
- Thread Pools

| Content

✓ Thread

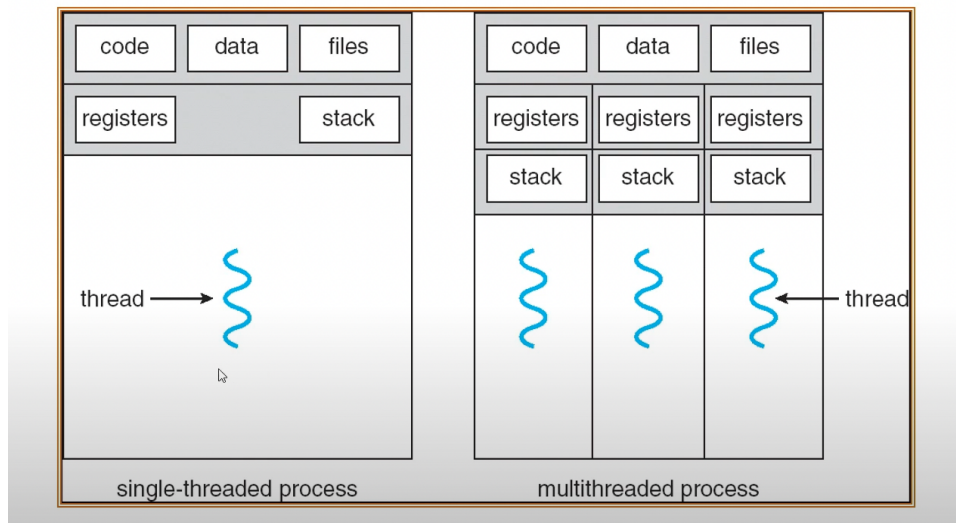
- the purpose of thread is the same as the process in that we need more than 1 portion of the program run independently at the same time.

- **Thread:** A portion of a program that can run independently of other portions.
 - A thread is a flow of execution through the process's code with its own program counter, system registers, and stack.
 - The **heavyweight process** which owns the resources becomes a more passive element. Thread is also called **lightweight process**.
 - Thread becomes the element that uses the CPU and is scheduled for execution
 - Swapping threads is less time consuming than swapping processes
- Multithreaded applications programs can have several threads running at one time with the same or different priorities

- Thread must have in own value like process, the main difference between thread and process is thread use less resources that process → sometimes, it called lightweight process.
- When we use thread, thread becomes the element that uses the CPU, OS choose thread to run in CPU
- Swapping thread (Context switch between thread) is less time consuming that swapping processes.
- Multithreaded applications programs is mean the program that can have several threads running at the same time. And the thread in the same program can have same priorities or different priorities.
 - OS can choose the higher priorities thread to run first.

 **Why create a thread use less resources than create a process?**

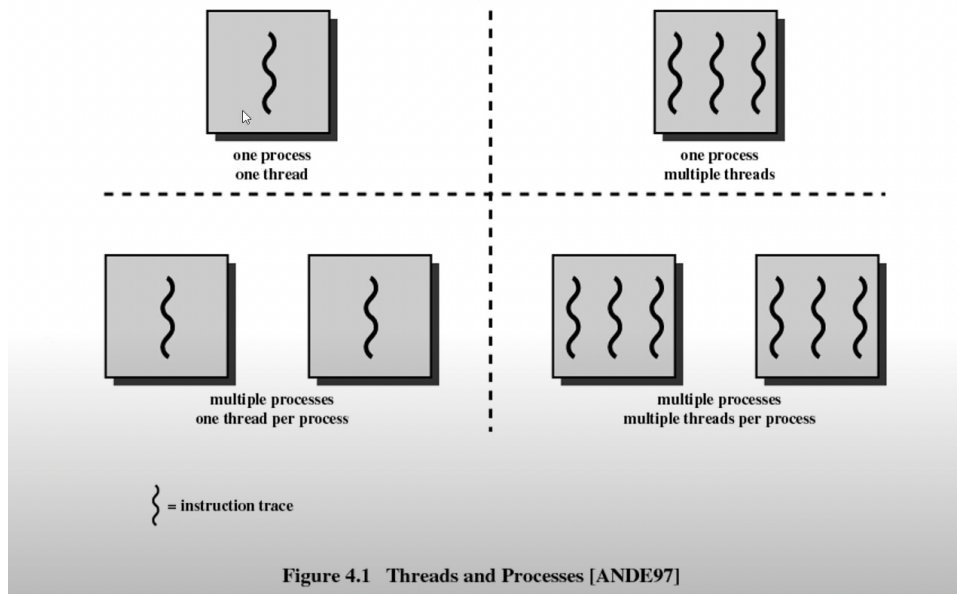
Single and Multithreaded Processes



- on the left hand side → system that one process support only one thread. So, If we need to have more than 1 portion of program to run at the same time, we need to create child process of this process.
 - create a child process → everything from parent process will get copy to child
- on the right hand side → system that support multithread process (in 1 process, you can create more than 1 thread)
 - from picture, each thread has it own register and stack, but they can share code, data, and files of the process **THIS IS WHY CREATE THREAD USE LESS RESOURCE THAN A PROCESS**

Categories of OS (Based on Thread)

Categories of OS (Based on Thread)



1. one process one thread

- OS that support 1 process at a time, and in a process, it has only 1 thread
 - ex. DOS

2. one process multiple threads

- you can run 1 process at a time, but in a process can have many thread
 - ex. JVM

3. multiple processes one thread per process

- the system can support multiple processes, but each process can have only 1 thread
 - ex. very old unix system

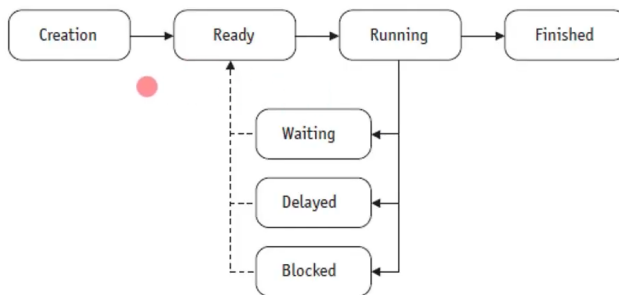
4. (Modern OS) multiple processes multiple threads per process

- support multiple processes, and each process can have multiple threads

Thread States

- very similar to process state

Thread States



(figure 6.6)

A typical thread changes states from *READY* to *FINISHED* as it moves through the system.

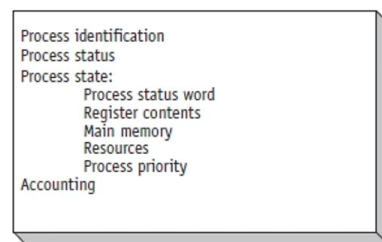
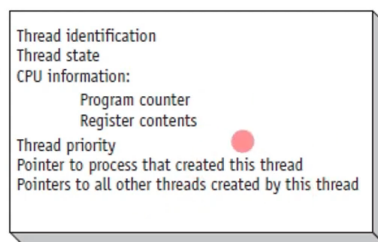
- Creation → similar to the new state
- Ready state → the same where the thread that are ready to run is in this state
- Running state → the same where the thread in this state running, using CPU
- Finished state → the same as terminated state
- Waiting, Delayed, Blocked → all of this are actually the waiting state of a process
 - Waiting → wait for some event to occur
 - Delayed → may call sleep function to slow down its work for some unit of time
 - Blocked → maybe doing the synchronization by some other thread

Thread Control Block

- Each thread must have thread control block

Thread Control Block

(figure 6.7)
Comparison of a typical Thread Control Block (TCB) vs. a Process Control Block (PCB) from Chapter 4.



- In thread control box, it has pointer to process control box (process that created this thread)

Types of Threads

There are two general types of threads:

- User-level threads (ULT)
- Kernel-level threads (KLT).

- User-level threads

User Threads

- Thread management done by user-level threads library
- Three primary thread libraries:
 - POSIX Pthreads
 - Win32 threads
 - Java threads
 - Generally implemented using a thread library available on host OS.
- Using this library, the application invokes the appropriate functions for the various thread management tasks such as: creating, suspending, and terminating threads.

- Thread work at the user mode, user level
 - this thread are control by thread library that are run in user level
- POSIX Pthreads - available on many unix like platform
- Win32 threads - thread library on window system
- Java threads - thread run on JVM
 - if JVM run on window, it may use Win32 threads
 - if JVM run on linux, it may use Pthread
- Kernel-level threads

Kernel Threads

- Supported by the Kernel
 - Examples
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X
- Thread that run in kernel mode (this thread control by OS)
 - Almost all modern OS support the idea of thread on kernel level.

Multithreading Model

when we run program using user level thread library → actually the thread we create is at user level, but the thread that must work for us must be the thread at kernel level

- there must be the mapping between user level thread and kernel level thread

3 Types of Mapping

Multithreading Models

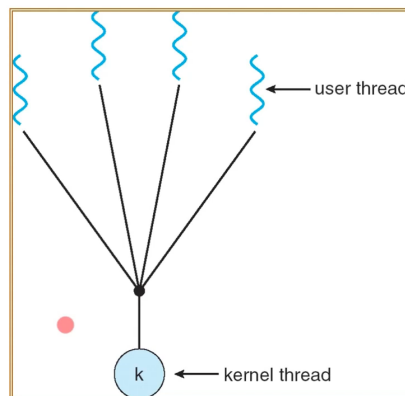
- Many-to-One
- One-to-One
- Many-to-Many

1. Many-to-One

Many-to-One

- Many user-level threads mapped to single kernel thread
- Examples:
 - Solaris Green Threads
 - GNU Portable Threads

Many-to-One Model



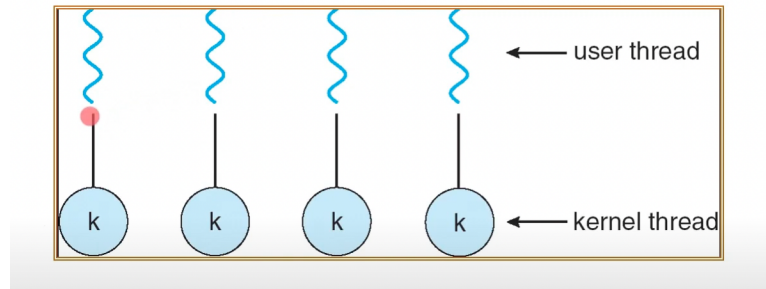
- mapping many user level thread to one kernel thread
- Actually this model is used when an OS support multiple processes, but each process has only single thread.
- The user thread not work truly concurrency, because the thread that work for them is 1 kernel thread.
- This model do not support full concurrency
- This model is used by very old OS that don't support multiple thread at kernel level

2. One-to-One

One-to-One

- Each user-level thread maps to kernel thread
- Examples
 - Windows NT/XP/2000
 - Linux
 - Solaris 9 and later

One-to-one Model



- mapping one user level to one kernel level
- modern OS support this model
- one user level map to one kernel level → this model support full concurrency → each thread has it own kernel level → it require more resources.
- You can't create user level thread, if the system said that I don't have enough resource to create kernel thread for you
- if thread go to waiting state, kernel level has to go to waiting state as well

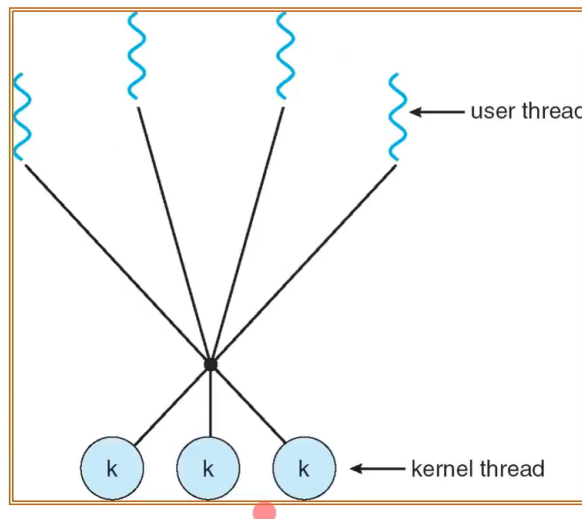
3. Many-to-Many

Many-to-Many Model



- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows NT/2000 with the *ThreadFiber* package

Many-to-Many Model



- mapping many user level to many kernel level.
- This model can't be consider the compromise of 1. and 2. above
 - it support better concurrency than many to one
 - the idea is as long as this 4 user thread don't need to work at a same time, and if we have 3 kernel level → it can work concurrency. But if all work at the same time → some thread has to wait
 - less resources that 1 to 1 model (no need to do 1 to 1 mapping), can use resources more efficient than one to one model.
- if one user thread go to waiting state, the kernel level can still work for 3 thread
- Problem in this model: Complexity in management that cost the OS performance
 - that why many OS implement one-to-one rather than many-to-many.

✓ Thread Library

- Pthreads

Pthreads

- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
 - API specifies behavior of the thread library, implementation is up to development of the library
 - Common in UNIX operating systems (Solaris, Linux, Mac OS X)
- (user library tread library)
 - available in many unix based OS like linux and MacOS

Pthreads

```
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */
main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of attributes for the thread */
    pthread_attr_init(&attr); /* get the default attributes */
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* now wait for the thread to exit */
    pthread_join(tid, NULL);
    printf("sum = %d\n", sum);
}
void *runner(void *param) {
    int upper = atoi(param);
    int i;
    sum = 0;
    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            sum += i;
    }
    pthread_exit(0);
}
```

- int sum → share
- for the thread program, we need to specify the program that the thread will be executed when it is created → in this case, we said that we want our thread to call runner function
 - line 2 → prototype of function runner
 - actually → the name doesn't have to be runner but the prototype format must be like this

- return void *, receive void*
 - we can think of void * as generic type pointer
- this program require command line parameter
- tid → stores thread id
- attr → set of attributes for the thread
- then we call function → to get default attribute
- pthread_create(&tid, &attr, runner, argv[1])
 - &tid → thread id, if successfully
 - &attr → use default attribute
 - runner → name of function that we want the thread to executed
 - argv[1] → command line parameter when we get when run the program, this will be pass to runner function parameter

after parent thread finish creating the child thread

- pthread_join(tid, NULL) → similar to wait pid, this parent thread wait for the thread this thread id, NULL (dont care about return value from the child thread)
- runner function
 - int upper = atoi(param) → use to convert string to int, store result to upper
 - find summation from 1 to number user enter
 - sum variable is share

prof version:


```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int sum; /* this data is shared by the thread(s) */
6  void *runner(void *param);
7  int main(int argc, char *argv[]) {
8      pthread_t tid;
9      pthread_attr_t attr;
10     if (argc != 2) {
11         fprintf(stderr, "usage: pthread <integer value>\n");
12         exit(1);
13     }
14     if (atoi(argv[1]) < 0) {
15         fprintf(stderr, "number must be >=0\n");
16         exit(2);
17     }
18     /* each thread has a set of attributes including stack size
19        and scheduling information .
20        In this example, we will not set the information, therefore,
21        we use the default attributes */
22     /* get the default attributes */
23     pthread_attr_init(&attr);
24     /* create the thread */
25     pthread_create(&tid, &attr, runner, argv[1]);
26     /* now wait for the thread to exit */
27     printf("I am mother thread, I will wait for my child thread\n");
28     pthread_join(tid, NULL);

```

- check if parameter == 2
- the parameter that pass to program must be string of digit only

```

sarun@sarun-tuf: ~/os/sourcecode
sarun@sarun-tuf:~/os/sourcecode/pthread$ gcc -o pthread pthread.c -l pthread
sarun@sarun-tuf:~/os/sourcecode/pthread$ ./pthread 5
I am mother thread, I will wait for my child thread
I am child thread, I am calculating
from my child sum = 15
sarun@sarun-tuf:~/os/sourcecode/pthread$

```

- child → calculate result

Java Threads

Java Threads

- Java threads are managed by the JVM
- Java threads may be created by:
 - Extending Thread class
 - Implementing the Runnable interface
- Extending Thread class

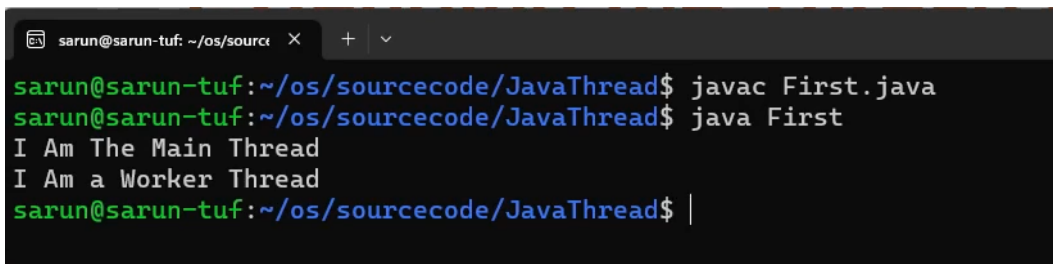
Extending the Thread Class

```
class Worker1 extends Thread
{
    public void run() {
        System.out.println("I Am a Worker Thread");
    }
}

public class First
{
    public static void main(String args[]) {
        Worker1 runner = new Worker1();
        runner.start();

        System.out.println("I Am The Main Thread");
    }
}
```

- we need to overwrite run method in java thread class
 - this run method will be call when our thread start working
 - the thing that it want the thread to do must be put inside run method
- when start method is called → the new thread is created and wait for it chance to run
 - if it allow to run by JVM, it will execute the run method.
- after runner.start() → we have 2 thread, main thread and worker thread.



```
sarun@sarun-tuf: ~/os/sourcecode/JavaThread$ javac First.java
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java First
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ |
```

output order is non-deterministics (depends of OS)

- Implement the runnable interface

The Runnable Interface

```
public interface Runnable
{
    public abstract void run();
}
```

- in this interface, it has 1 abstract method that our class need to be implement

Implementing the Runnable Interface

```
class Worker2 implements Runnable
{
    public void run() {
        System.out.println("I Am a Worker Thread ");
    }
}

public class Second
{
    public static void main(String args[]) {
        Runnable runner = new Worker2();
        Thread thrd = new Thread(runner);
        thrd.start();

        System.out.println("I Am The Main Thread");
    }
}
```

- the thing you want thread to do → out in run()

```
1
2 //Second.java
3
4
5 public class Second
6 {
7     public static void main(String args[]) {
8         Runnable runner = new Worker2();
9
10        Thread thrd = new Thread(runner);
11
12        thrd.start();
13
14        System.out.println("I Am The Main Thread");
15    }
16 }
17
```

```

1
2 // Worker2.java
3
4 class Worker2 implements Runnable
5 {
6     public void run() {
7         System.out.println("I Am a Worker Thread ");
8     }
9 }
10

```

```

sarun@sarun-tuf: ~/os/sourcecode/JavaThread$ code .
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ javac Second.java
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java Second
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java Second
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java Second
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java Second
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java Second
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java Second
I Am The Main Thread
I Am a Worker Thread
sarun@sarun-tuf:~/os/sourcecode/JavaThread$

```

- ความจริงเลือกอันไหนก็ได้ แต่ถ้า software dev point of view → implement runnable interface is better → avoid using inheritance
- situation force you to use implement runnable
 - when your class already inherit from super class (java doesn't support multiple class inheritance).

```

class Bar extends Foo {
}

```

```

class Bar extends Foo implements Runnable{
}

```

📌 Joining Threads

- if parent thread need to wait for child thread, use ,join()

Joining Threads

```
class JoinableWorker implements Runnable
{
    public void run() {
        System.out.println("Worker working");
    }
}

public class JoinExample
{
    public static void main(String[] args) {
        Thread task = new Thread(new JoinableWorker());
        task.start();

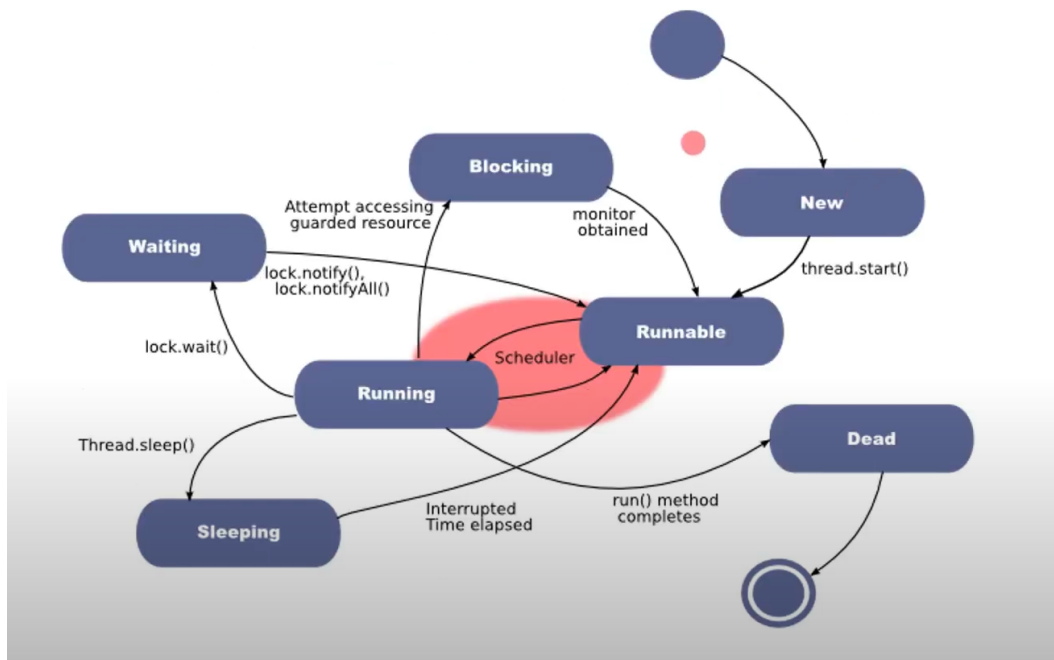
        try { task.join(); }
        catch (InterruptedException ie) { }

        System.out.println("Worker done");
    }
}
```

- in this program, worker done will be printed after worker working.

📌 Java Thread States

Java Thread States



- new is the same as new state in process
- when start method is called → thread will be in runnable state

- runnable → ready state
- if JVM choose thread to run → thread be in running state
 - running state → execute statement in run method
- if run method complete → go to dead state
- if sleep() get called → sleeping state
- Waiting and Blocking ,we discuss more detail alter when talk about synchronization

Thread Cancellation

- thread can be terminated before it finished.

Thread Cancellation

- Terminating a thread before it has finished
 - Two general approaches:
 - **Asynchronous cancellation** terminates the target thread immediately
 - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
 - the parent thread can cancelled the child thread
- 2 general approach**
- Asynchronous cancellation
 - the target tread will be terminated immediately
 - Deferred cancellation
 - allow target thread to work until the point that it safe to terminate

Java Support Deferred cancellation

Thread Cancellation

```
Thread thrd = new Thread (new InterruptibleThread());  
Thrd.start();
```

...

```
// now interrupt it  
Thrd.interrupt();
```

- Main thread start the child thread
- call interrupt method to interrupt the child thread

Thread Cancellation (Cont.)

```
public class InterruptibleThread implements Runnable  
{  
    public void run() {  
        while (true) {  
            /**  
             * do some work for awhile  
             */  
  
            if (Thread.currentThread().isInterrupted()) {  
                System.out.println("I'm interrupted!");  
                break;  
            }  
        }  
    }  
  
    // clean up and terminate  
}
```

- the child thread can work until it finish the first part (until check)
 - if it interrupt → terminates

Example

```

1  /**
2   * This program interrupts the Interruptible Thread
3   *
4   * @author Gagne, Galvin, Silberschatz
5   * Operating System Concepts with Java - Sixth Edition
6   * Copyright John Wiley & Sons - 2003.
7   */
8  public class Interrupter
9  {
10     public static void main(String[] args) {
11         Thread worker = new Thread (new InterruptibleThread());
12         worker.start();
13
14         // now wait 3 seconds before interrupting it
15         try { Thread.sleep(3000); } catch (InterruptedException ie) { }
16
17         worker.interrupt();
18     }
19 }
20

```

Interrupter → Main thread

- create child tread
- start
- wait for 3 second (sleep method) (unit → mili second)
- after wait for 3 second → called interrupt to interrupt the child thread

```

J Interrupter.java  J InterruptibleThread.java X
J InterruptibleThread.java
8  /**
9   * Operating System Concepts with Java - Sixth Edition
10   * Copyright John Wiley & Sons - 2003.
11   */
12  public class InterruptibleThread implements Runnable
13  {
14     /**
15     * This thread will continue to run as long
16     * as it is not interrupted.
17     */
18     public void run() {
19         while (true) {
20             System.out.println("I am working");
21
22             if (Thread.currentThread().isInterrupted()) {
23                 System.out.println("I'm interrupted!");
24                 break;
25             }
26         }
27         // clean up and terminate
28     }
29 }
30

```

InterruptibleThread → Child thread

- after print → check if it interrupt

- if so, print i'm interrupt and break out the loop

```
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I am working
I'm interrupted!
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ |
```

print I'm working for 3 seconds and interrupt

Thread Pools

- to use thread more effectively → we can use the idea of thread pool

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool
- Thread pool mean we crate a number of thread in advanced and put them in a pool where they wait for working.
 - using this idea, we get service from a thread faster, because, instead of waiting for thread creation every time we need it, now we can use existing thread from thread pool.
 - And after that thread finish working for us, it go back to thread pool
 - Developer can decide how many thread they need for application → create them in advanced and put them in the pool.

Java provides 3 thread pool architectures:

Thread Pools

- Java provides 3 thread pool architectures:
 1. **Single thread executor** - pool of size 1.
 - `static ExecutorService newSingleThreadExecutor()`
 2. **Fixed thread executor** - pool of fixed size.
 - `static ExecutorService newFixedThreadPool(int nThreads)`
 3. **Cached thread pool** - pool of unbounded size
 - `static ExecutorService newCachedThreadPool()`

1. Single thread executor

- we create pool of size 1 → there only 1 thread in a pool

2. Fixed thread executor

- We specify how many thread that we need in the pool
 - for example, pool of size 10 (specify 10 as parameter)

3. Cache thread pool

- if we use this idea, the system will automatically increase the pool size if we need more thread

Thread Class

Thread Pools

A task to be serviced in a thread pool

```
public class Task implements
Runnable {
    public void run() {
        System.out.println("I am working
on task");
    }
}
```

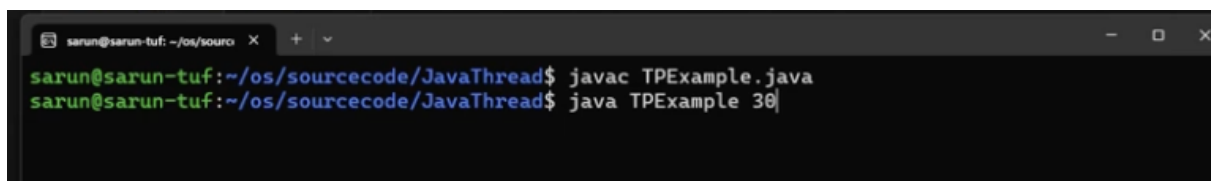
- this is thread class, we print I am working on task

Thread Pools

```
import java.util.concurrent.*;

public class TPExample {
    public static void main(String[] args) {
        int numTasks = Integer.parseInt(args[0].trim());
        ExecutorService pool =
            Executors.newCachedThreadPool();
        for(int i =0; i < numTasks; i++) {
            pool.execute(new Task());
        }
        pool.shutdown();
    }
}
```

- Main thread
 - we create thread using the idea of unbounded size
 - we ask user to enter the number of task that he/she want
 - in for loop, we call execute method and said we need to work with this thread class
 - we don't have to call start method → because the pool of thread already created.
 - we just said that we need to use 1 thread in a pool to work for us and work with the thread class.



```
sarun@sarun-tuf: ~/os/source x + v
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ javac TPExample.java
sarun@sarun-tuf:~/os/sourcecode/JavaThread$ java TPExample 30
```

